WAYS OF TESTING:

There are 2 ways of Testing:

1. MANUAL TESTING

2. AUTOMATION TESTING

1. MANUAL TESTING: Manual Testing is a process, in which all the phases of STLC (Software Testing Life Cycle) like

Test planning, Test development, Test execution, Result analysis, Bug tracking and Reporting are accomplished

successfully and manually with Human efforts.

DRAWBACKS:

1. More no of human resources are required.

2. Time consuming.

3. Less accuracy.

4. Tiredness.

5. Simultaneous actions are almost impossible.

6. Repeating the same task again and again in same fashion is almost impossible.

2. AUTOMATION TESTING: Automation Testing is a process, in which all the drawbacks of Manual Testing are

addressed properly and provides speed and accuracy to the existing testing process.

DRAWBACKS:

1. Automated tools are expensive.

2. All the areas of the application can't be tested successfully with the automated tools.

3. Lack of automation Testing experts.

Note: Automation Testing is not a replacement for Manual Testing, it is just continuation for Manual Testing.

Note: Automation Testing is recommended to be implemented only after the application has come to a stable

stage.

Is Automation Successful in Agile?

Agile is a term used for several iterative and incremental software development life cycle methods. Most popular methodologies include extreme programming, scrum etc which are implemented or said to be implemented nowadays by teams. The development cycles have become shorter, crisper with feedback and testing being conducted in these small sprints. The SCRUM model of agile follows iterative sprints, where a part of the product is shipped. And in this sprint, the part has to be developed and tested. With every sprint or iteration, the product is going to be built, the code is going to become larger but the time to test will remain the same. Automation is not a choice in agile methodology, it is a necessity.

All about Automation & Agile

- **Changing Requirement & Tests**

We need to be careful with automation when working in agile mode. We have to realize and understand that the product under test is constantly changing with every sprint, and our tests should not break down because of the changing test objects. So the test code should be kept separate from the UI of the application, and as much as a possible modular approach to code should be enforced. With every sprint, the test data will become obsolete, so separate code should be written to quickly generate the test data so that it can be utilized in the current sprint cycle.

- **Automation saves Time**

We are also looking at the integration of our test automation cycle to log bug directly to the bug tracking tool during execution, and at the same time mark the user story as the failure in that build, so we are looking for integration with an ALM tool. The regression tests form the ideal candidates for automation, and their count will increase with every sprint cycle. We also have to be careful about the time it is taking for automation to execute the test scenarios. We should apply dynamic waits, data generation without UI usage, modular code approach to our automation scripts.

- **Early Automation**

We also need to understand that we do not require a stable build, for automation to begin. Automation activity can get started with the start of the product coding. Identify the common scenarios, data creation prerequisites, and automate them first. Then build a framework, lay down the guidelines for the automation code, so that with every sprint, we can add to this suite and execute the tests to test the build for the regression scenarios.

- **Continuous Integration**

We also need to look at the build creation process and acceptance of it for the testing. In the agile model, we cannot afford for the build to be made after 3 weeks of a code being written. We need nightly builds which are created automatically and tested against a test automation suite for build acceptance for testing. This process falls under the umbrella of Continuous Integration. CI is also a must with agile. Tools like Jenkins, Hudson provide many triggers for build creation process and we can attach our test automation suite with the process against which the build will be tested as soon as it is made. So by next day morning, the developer will know whether the changes he made yesterday affected the build in a positive way or has broken the build.

- **Build frequency in Agile**

In agile information is the key. We need information, quick. We need to act on that information and derived decision fast. The feedbacks from stakeholders are available in a bat of an eyelid. Automation is not to replace manual testing. We cannot. But it is required to act as an aid, to help derive that decision faster, by taking less time and cost. We can also make use of our manual testing team to better work than the redundant work of regression testing required for every build release. We need to understand that the frequency of these build releases in agile model increases because we have shorter life cycles, called as sprints.

- **Good Planning**

Like with all good and necessary things a word of caution is attached, it is also true in this case. Though automation is essential, its implementation is full of caveats. We do not want that with every sprint cycle our automation suite is breaking because of the code change, UI change and the entire sprint cycle which is anyways shorter than the traditional cycle is spent on fixing it. So we have to be careful, start early and start right the first time when implementing automation in agile.

- **Automation has wide scope in Agile**

You will note that the automation is not just to shorten the testing process, automation is also needed for the build making process, and automation is also required for bug reporting process. We have to respect that agile method requires information at a speed, and we cannot hinder it by doing processes the manual way when an alternative in terms of automating them is available. The tools for CI, ALM, test automation make an integral part of the agile way of producing products.

If reading the above information, you still have doubt as if it is worth to do automation in agile, try doing agile without automation, it will not be true to the spirit of agile. It is an obligatory activity. It assists both the developers and testers, giving them the necessary support which is required to handle the sprints, and the spirit of agile.

Agile Scrum for Testers

What is agile and scrum?

- **Agile meaning:** Able to move quickly and easily.

- **Scrum meaning:** a Rugby play

Agile Scrum:

- It is an iterative and incremental **agile software** development framework for managing product development.

- It defines a flexible product development strategy where a development team works as a unit to reach a common goal.

- It is an **Incremental** and **Iterative** model.

- It is a self- organized and focused team.

- No huge document, rather have precise and to point stories.

- Close communication with user Representative.

- Have definite time lines of 1 week to 4 weeks

- Instead of doing everything at a time scrum does a little of everything at a given interval.

- Resources capability and availability is considered before committing any thing.

Why Agile Scrum?

- Incremental approach breaks complex projects down into simpler mini-projects
- Accommodates change easily
- Improves ROI through frequent and regular delivery of value to the business
- Increased business involvement and satisfaction
- Increased visibility (progress, obstacles, risks, etc) Reasons to use Agile
- Lower development risk, higher quality, less defects
- Shorter cycles produce working software and incremental product quickly
- Progress measured by running tested software
- Early and regular process improvement driven by frequent inspection

<u>Agile Scrum overview:</u>

Advantages

- We can save time and cost of the project.
- The quality can be ensured because each and every sprint will be tested multiple times.
- The requirements change can be accepted at any level of the project maintenance.
- All are participating in Scrum meting so that transparency can be maintained.
- Each and every sprint we are delivering to the client so we can maintain the customer's satisfaction and we can avoid delivery risk of the project.

Scrum Terminology

- **Roles** : Product Owner, Scrum Master, Team
- **Ceremonies** : Sprint Planning, Sprint Review, Sprint Retrospective, & Daily Scrum Meeting
- **Artifacts** : Product Backlog, Sprint Backlog, and Burn down Chart

Product Owner

- Define the features of the product
- Decide on release date and content
- Be responsible for the profitability of the product (ROI)
- Prioritize features according to market value
- Adjust features and priority every iteration, as needed
- Accept or reject work results.

The Scrum Master

- Represents management to the project
- Responsible for enacting Scrum values and practices
- Removes impediments
- Ensure that the team is fully functional and productive

- Enable close cooperation across all roles and functions
- Shield the team from external interferences

Scrum Team

- Typically 5-10 people
- Cross-functional
- QA, Programmers, UI Designers, etc.
- Members should be full-time

Meetings

- Sprint Planning Meeting
- Daily Scrum
- Sprint Review Meeting
- Sprint Retrospective Meeting

Daily Scrum Meeting

- Parameters

- Daily
- 15-minutes
- Stand-up
- Not for problem solving
- Three questions:
- What did you do yesterday
- What will you do today?
- What obstacles are in your way?

Sprint Review Meeting

- Team presents what it accomplished during the sprint
- Typically takes the form of a demo of new features or underlying architecture

- Informal
- 2-hour prep time rule
- Participants
- Customers
- Management
- Product Owner
- Other engineers

Sprint Retrospective Meeting

- Scrum Team only
- Feedback meeting
- Three questions
- What went well
- What went wrong

- What can be done(Improvement)

- Don’t skip for the first 5-6 sprints!!!

Agile Methodology

What is Agile Methodology?

- It is an Iterative and Incremental Approach.
- Iterative means same process repeating again and again.(The process keeps on repeating).
- Incremental means, modules/features keep on adding on top of existing software.
- Agile is Iterative and Incremental model where requirements keeps on changing.
- As a company we should be flexible to accept requirements change, develop, test and finally release a peace of working software within short span of time.
- There will be good communication between Customer, Business Analyst, Developers & Testers.
- The Goal of the agile model is the customer satisfaction by delivering the piece of the software to the customer within short span of time.
- Agile Testing is type of testing where we follow the agile principles.

Advantages:

- Requirement changes are allowed in any stage of development (or) We can accommodate Requirement changes in the middle of development.
- Releases will be very fast(Weekly)
- Customer no need to wait for long time.
- Good communication between team.
- It is very easy model to adopt.

Disadvantage:

- Less focus on design and documentation since we deliver software very faster.

What is Scrum?

Scrum is a framework through which we build software product by following Agile Principles.

Scrum includes group of people called as Scrum team.

- Product Owner
- Scrum Master
- Dev Team
- QA Team

Product Owner :

- Define the features of the product
- Prioritize features according to market value
- Adjust features and priority every iteration, as needed
- Accept or reject work results.

Scrum Master:

- The main role is facilitating and driving the agile process.

Developers and QA:

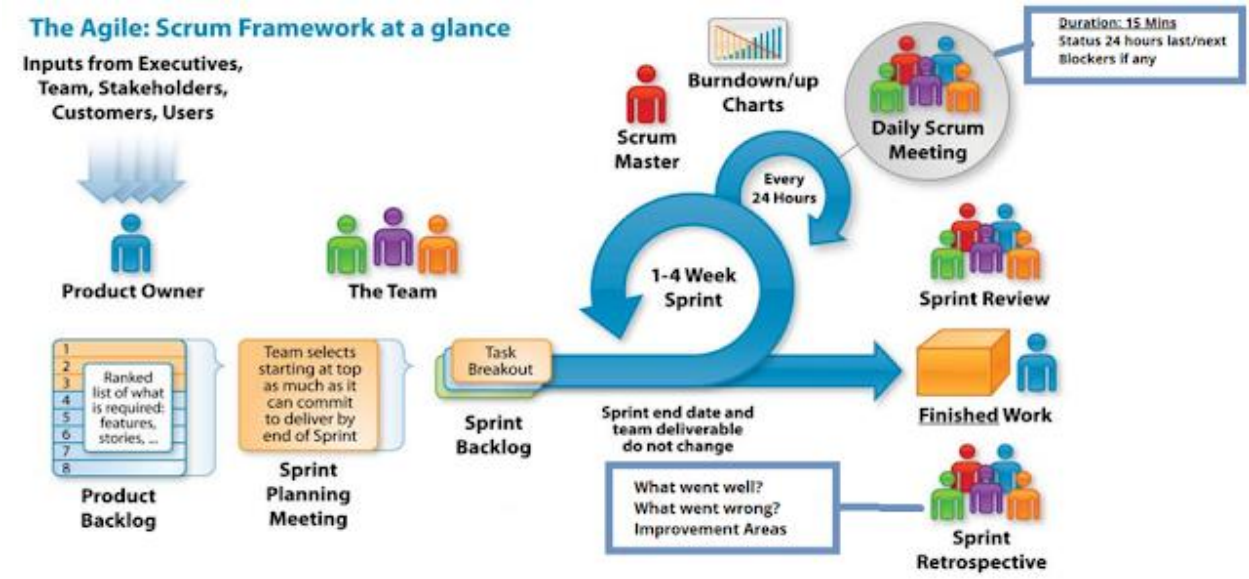
- Develop and Test the software.

Agile Vs Scrum

Agile:
Focus: Agile is an approach to project management and product development that emphasizes flexibility and customer satisfaction.
Key Principles: It values collaboration, adaptability, and delivering small, functional pieces of a project regularly.
Benefits: Allows for changes in project requirements, encourages customer feedback, and promotes a collaborative team environment.

Scrum:
Type of Agile Framework: Scrum is one of the specific frameworks within the broader Agile methodology.
Roles: In Scrum, there are defined roles - Scrum Master, Product Owner, and Development Team.
Artifacts: It uses specific artifacts like the Product Backlog, Sprint Backlog, and Increment to manage and deliver work.
Events: Scrum includes specific events or ceremonies like Sprint Planning, Daily Standup, Sprint Review, and Sprint Retrospective.

Scrum Terminology



User Story : A Feature/module in a software

Epic : Collection of user stories.

Product backlog : Contains list of user stories. Prepared by product owner.

Sprint : Period of time to complete the user stories, decided by the product owner and team, usually 2-4 weeks of time.

Sprint planning meeting: Meating conducts with the team to define what can be delivered in the sprint and duration.

Sprint backlog : List of committed stories by Dev/QA for specific sprint.

Scrum meeting : Meating conducted by Scrum Master everyday 15 mins. Called as Standup meeting.

1. What did you do yesterday?
2. What will you do today?
3. Are there any impediments in your way?

Sprint retrospective meeting : Review meeting after completion of sprint. The entire team, including both the ScrumMaster and the product owner should participate.

Story point : Rough estimation of user stories, will be given by Dev & QA in the form of Fibonacci series.

Burndown chart : Shows how much work remining in the sprint. Maintained by the scrum master daily.

DoR & DoD

- **Definition of Ready (DoR):**
- Preparation: Ensures tasks are well-prepared before starting work.
- Clarity: Describes what needs to be done, making sure everyone understands the plan.
- Timing: Decided before starting a task or user story during planning.
- Owner: Managed by the task planner or team lead.
- Adjustable: Can be tweaked as needed during planning.
- **Definition of Done (DoD):**
- Completion: Declares when a task or user story is considered finished.
- Criteria: Lists specific standards that must be met for completion.
- Timing: Decided at the beginning of the project or sprint.
- Shared Responsibility: Owned by the entire team, including developers and testers.
- Consistency: Should remain constant during the sprint; changes considered for future sprints.

Agile Meetings

1) Sprint Planning:

- Attendees: Entire team (developers, testers, product owner).
- When: At the beginning of each sprint.
- Duration: Typically 1-2 hours.
- Purpose: Plan and prioritize tasks for the upcoming sprint.

2) Daily Standup (Daily Scrum):

- Attendees: Entire team.
- When: Daily, preferably in the morning.
- Duration: 15 minutes or less.
- Purpose: Share updates on work, discuss challenges, and align for the day.

3) Sprint Review:

- Attendees: Team, stakeholders, product owner.
- When: At the end of each sprint.

- Duration: 2-4 hours.
- Purpose: Showcase completed work, gather feedback, and discuss what's next.

4) Sprint Retrospective:

- Attendees: Team members.
- When: At the end of each sprint, after the sprint review.
- Duration: 1-2 hours.
- Purpose: Reflect on the sprint, discuss what went well and what could be improved, and plan for adjustments.

5) Backlog Grooming (Refinement):

- Attendees: Product owner, Scrum Master, development team.
- When: As needed between sprints.
- Duration: Typically 1-2 hours.
- Purpose: Review and refine the product backlog, ensuring items are well-defined and ready for upcoming sprints.

Story point

A story point is a unit of measure used to estimate the difficulty or complexity of a task or user story.
Estimating a user story in Agile involves assigning it a story point value, and teams often use the Fibonacci sequence (1, 2, 3, 5, 8, 13, etc.)

Estimating a story using story point

User Story: "As a user, I want to be able to log in to the application using my email and password."

Estimation Process:

- **Understand the User Story:**
- The team discusses the user story to ensure everyone understands what's required. Logging in with email and password seems straightforward.
- **Compare Complexity:**
- The team compares this user story to a reference story. Let's say the reference story is a simple one-point story, like "displaying a welcome message."
- **Use Relative Sizing:**
- Team members discuss and agree that logging in is a bit more complex than displaying a welcome message but not significantly more complex. They decide to assign it a story point value of 2.
- **Fibonacci Sequence:**

- The team considers whether the complexity is closer to 2 or 3 in the Fibonacci sequence. After discussion, they agree that 2 is a more accurate representation.
- **Team Consensus:**
- The team discusses any differing opinions. If someone initially suggested 3, they might discuss why they thought it was more complex. After a brief discussion, the team reaches a consensus, and everyone agrees on 2 story points.
- **Record the Estimate:**
- The team records the estimate of 2 story points for the "log in" user story. This estimate will be used for planning and prioritizing in the upcoming sprint.

1 Story Point: Typically takes a few hours to complete (half a day).
2 Story Points: Could take a day to a day and a half.
3 Story Points: Might take two days.
5 Story Points: Could take around three days.
8 Story Points: A larger task, likely taking a week.
13 Story Points: A significant effort, possibly spanning multiple weeks.

Burn-Down Charts

There are four popularly used burn down charts in Agile.

- **Product burndown chart :** A graph which shows how many Product Backlog Items (User Stories) implemented/not implemented.
- **Sprint burndown chart :** A graph which shows how many Sprints implemented/not implemented by the Scrum Team.
- **Release burndown chart :** A graph which shows List of releases still pending, which Scrum Team have planned.
- **Defect burndown chart :** A graph which shows how many defects identified and fixed.